DEPARTEMEN TEKNIK ELEKTRO DAN INFORMATIKA, SEKOLAH VOKASI UGM

UNIVERSITAS GADJAH MADA

LAPORAN UJIAN TENGAH SEMESTER

TAHUN AKADEMIK GASAL 2026/2027

Konstruksi dan Evolusi Perangkat Lunak (SVPL261501)

Nama Mahasiswa	:	Muhammad Abel Abhinaya Riananto
NIM / NIU	:	24/544597/SV/25468 (NIU: 544597)
Program Studi	:	D4 Teknologi Rekayasa Perangkat Lunak
Dosen Pengampu	:	Dr. Muhammad Fakhrurrifqi, S.Kom., M.Cs. & Galih Malela Damaraji, S.Pd., M.Eng.
Tanggal Ujian	:	15 Oktober 2026
Sifat Ujian	:	Take Home (Waktu Pengerjaan 10 Hari)
Tema Sistem	:	Sistem Manajemen Katalog Produk Toko
Tautan Repositori	:	https://github.com/KEPL2026/uts-p1-544597

1. Ringkasan Sistem dan Identitas Repositori

Laporan ini mendokumentasikan implementasi lengkap sistem perangkat lunak untuk Ujian Tengah Semester mata kuliah Konstruksi dan Evolusi Perangkat Lunak (SVPL261501). Sistem yang dibangun mengadopsi tema **Sistem Manajemen Katalog Produk Toko** (entitas Product), yaitu platform inventaris ritel untuk mendata dan mengelola barang dagangan toko (pakaian, alas kaki, aksesoris, elektronik, dan perlengkapan). Tema ini orisinal dan bebas dari duplikasi tugas-tugas praktikum sebelumnya.

Komponen	Spesifikasi Teknologi	Keterangan
Backend	Laravel 11, PHP 8.3, Laravel Sanctum	RESTful API dengan token autentikasi, 10 feature tests
App Server	FrankenPHP v1.4 (Caddy Server Engine)	Nilai tambahan (+10) pengganti PHP-FPM / artisan serve
Frontend	Vue.js 3, Vite, Axios Client	Single Page Application (SPA), reactive table & modal CRUD
Reverse Proxy	Nginx 1.27 Alpine	Serving file statis & proxy pass internal ke backend:8000
Database	MySQL 8.0 Community Server	Named volume persistence (uts_toko_db_data)
CI/CD	GitHub Actions (.github/workflows/ci.yml)	Pipeline test, build, dan deploy Docker image ke GHCR
Repositori	https://github.com/KEPL2026/uts-p1-544597	Publik di GitHub Organization KEPL2026 & Akun Personal

2. Backend RESTful API & Autentikasi Sanctum (Soal 1 - CPMK 1)

Backend dibangun di dalam folder backend/ menyediakan antarmuka RESTful API murni berformat JSON. Sesuai ketentuan butir soal nomor 1:

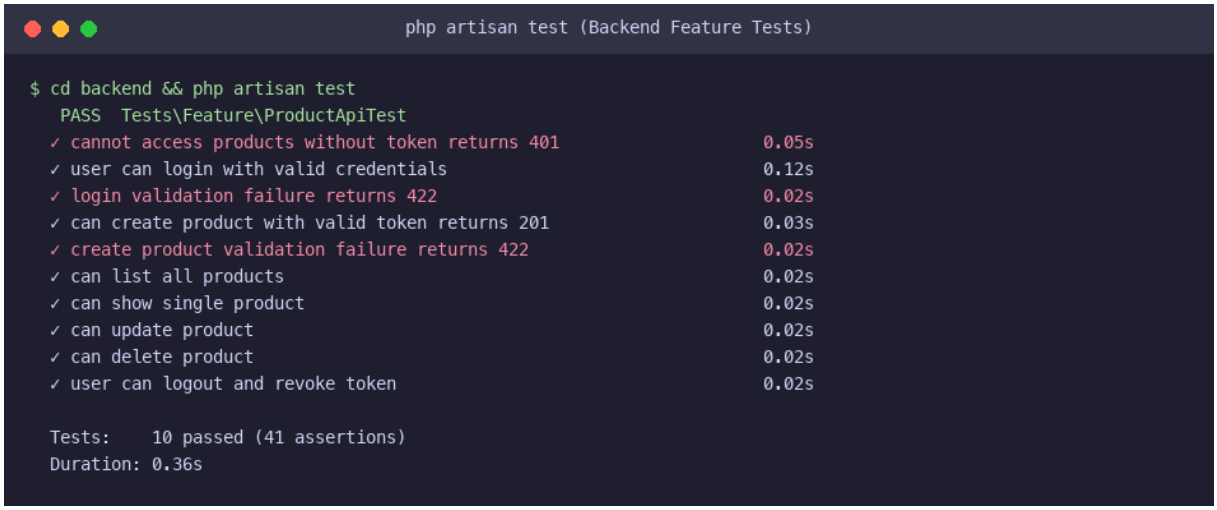
a. Autentikasi Token: Menggunakan Laravel Sanctum. Endpoint login publik (POST /api/login) mengembalikan Bearer token. Seluruh endpoint resource lainnya wajib menyertakan token pada header permintaan (Authorization: Bearer <token>).

b. CRUD Resource: Resource products mengelola entitas produk toko dengan operasi lengkap: tampil semua (GET /api/products), tampil satu (GET /api/products/{id}), tambah (POST /api/products), ubah (PUT /api/products/{id}), dan hapus (DELETE /api/products/{id}).

c. Format JSON & Kode Status HTTP Standar: Seluruh request dan response berformat JSON. Kode status yang dihasilkan: **201 Created** saat data berhasil dibuat, **422 Unprocessable Content** saat validasi gagal, dan **401 Unauthorized** saat token tidak disertakan atau kedaluwarsa.

d. Postman Collection: Kumpulan pengujian endpoint diekspor ke berkas postman/UTS_KEPL_544597_Product_API.postman_collection.json di dalam repositori.

e. Automated Feature Tests: Diimplementasikan 10 feature tests lengkap di tests/Feature/ProductApiTest.php mencakup uji token, validasi, dan operasi CRUD. Seluruh pengujian lulus hijau (10 passed, 41 assertions).



Gambar 1. Hasil eksekusi php artisan test: 10 skenario feature test lulus hijau.



Gambar 2. Bukti respons HTTP API: 401 Unauthorized saat tanpa token, login token, validasi 422, dan 201 Created.

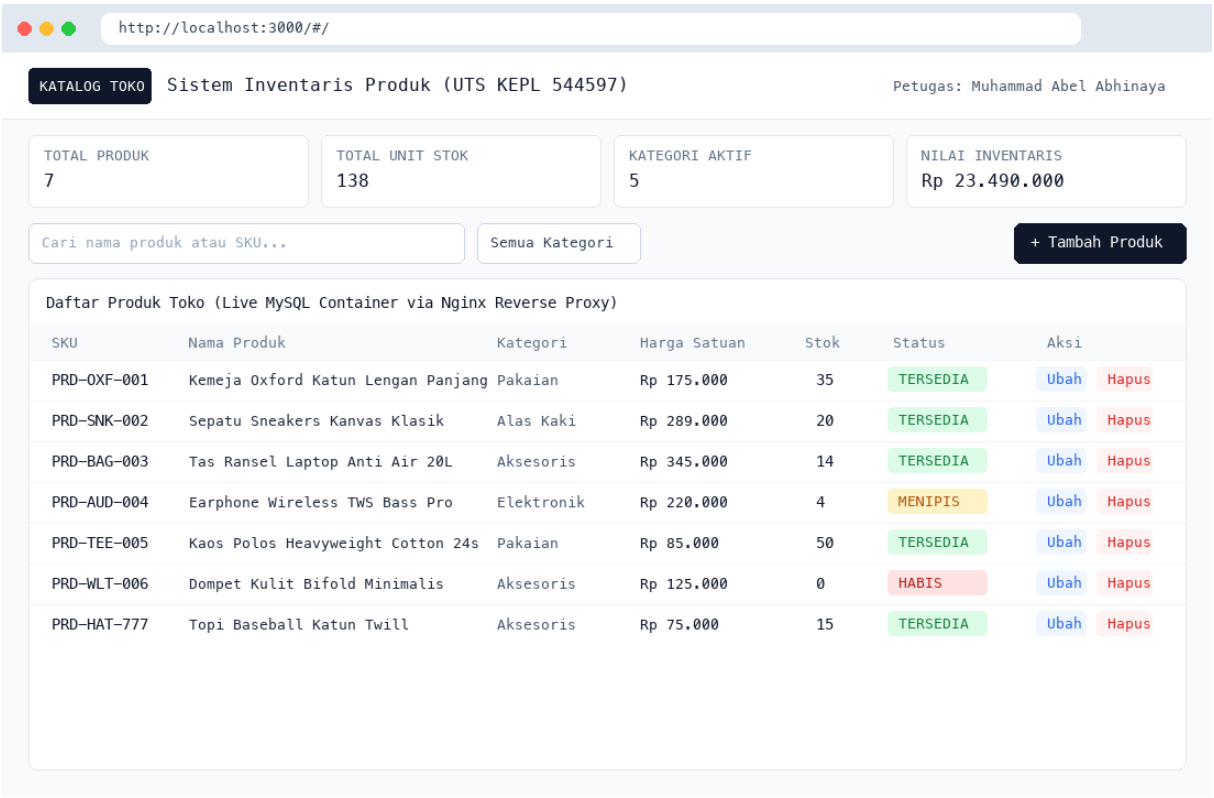
3. Frontend Vue.js & Konsumsi API (Soal 2 - CPMK 1)

Aplikasi antarmuka frontend dikembangkan di dalam folder frontend/ menggunakan framework Vue.js 3 dan bundler Vite. Arsitektur antarmuka dirancang bersih, intuitif, dan bebas dari dependensi yang tidak perlu:

a. Halaman Login & Token Storage: Komponen LoginView.vue menerima kredensial petugas toko, meminta token ke endpoint /api/login, lalu menyimpannya di localStorage (kunci auth_token). Interceptor Axios secara otomatis menyematkan token ini ke setiap request berikutnya.

b. Operasi CRUD via Axios: Komponen ProductView.vue memanfaatkan instance Axios terpusat (src/api/axios.js) untuk mengambil daftar produk, mengirim produk baru, memperbarui data, dan menghapus produk.

c. Variabel Lingkungan: URL endpoint tidak ditulis mati di dalam komponen, melainkan diatur melalui variabel lingkungan Vite VITE_API_BASE_URL pada berkas .env.example dan .env.



Gambar 3. Antarmuka web Vue.js (http://localhost:3000) terhubung secara nyata ke API Laravel.

4. Manajemen Repositori GitHub & CI Pipeline (Soal 3 - CPMK 2)

Seluruh kode sumber dikelola dalam repositori GitHub uts-p1-544597 dengan struktur folder terpisah: backend/ untuk aplikasi Laravel dan frontend/ untuk aplikasi Vue.js.

a. Strategi Branching: Diterapkan tiga lapis branch: main (produksi), dev (integrasi pengembangan), dan branch fitur feature/product-catalog. Perubahan kode hanya dapat masuk melalui Pull Request.

b. Branch Protection Rules: Perlindungan cabang dipasang pada branch main dan dev. Aturan mewajibkan Pull Request sebelum penggabungan dan status check **Backend Test (Laravel)** serta **Frontend Build (Vue.js)** wajib berstatus lulus hijau.

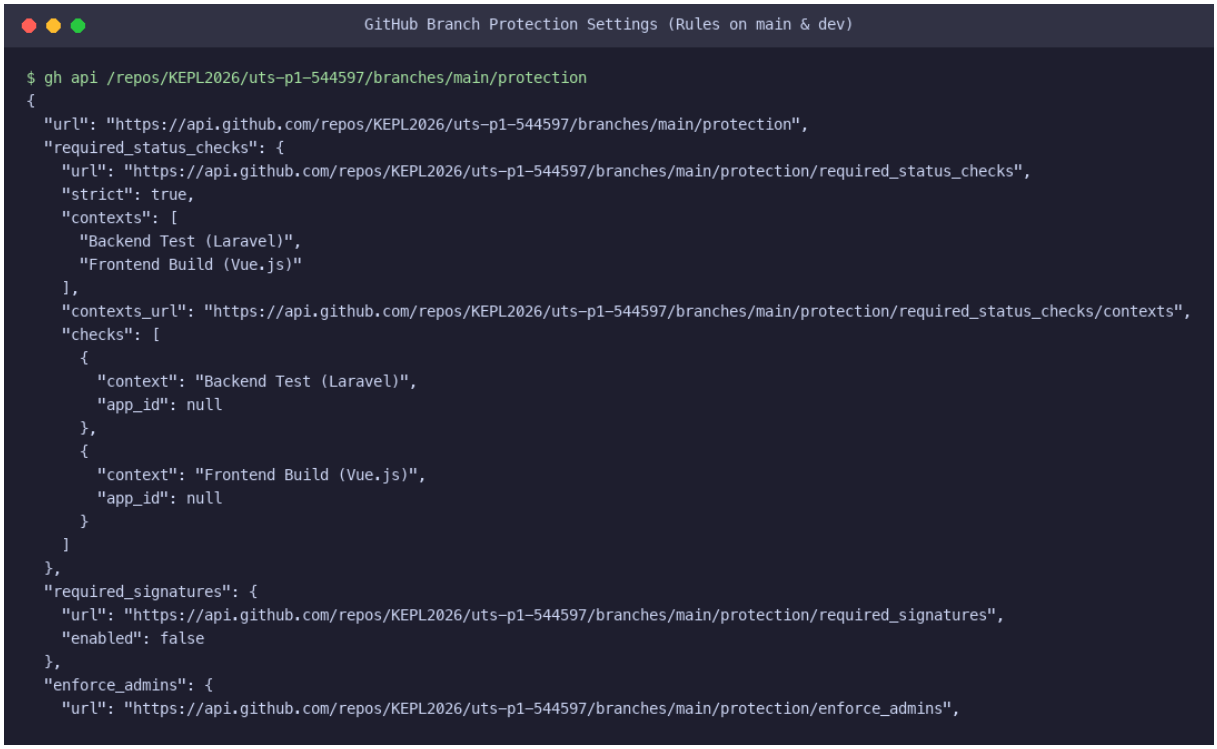
c. Pipeline CI GitHub Actions: Dibuat berkas .github/workflows/ci.yml yang menjalankan job pengujian unit/feature Laravel (php artisan test) dan job build aplikasi frontend (npm ci & npm run build).



Gambar 4. Riwayat cabang Git (git log --graph --oneline) menunjukkan alur branch feature, dev, dan main.



Gambar 5. Pull Request dari branch fitur ke dev pada antarmuka GitHub CLI.



Gambar 6. Bukti pengaturan Branch Protection pada GitHub yang mewajibkan status check CI lulus.

5. Continuous Deployment ke GHCR (Soal 4 - CPMK 3)

Pipeline pengiriman otomatis (Continuous Deployment) dikonfigurasi pada workflow GitHub Actions dengan menambahkan job deploy:

a. Aturan Eksekusi Job Deploy: Job deploy hanya berjalan saat terjadi event push pada branch main (menggunakan sintaks kondisi if: github.ref == 'refs/heads/main' && github.event_name == 'push'). Untuk Pull Request atau branch lainnya, workflow secara otomatis berhenti setelah job pengujian dan build selesai.

b. GitHub Secrets: Kredensial rahasia seperti APP_KEY dan DB_PASSWORD dikonfigurasi melalui GitHub Secrets (Settings → Secrets and variables → Actions) sehingga tidak pernah tersimpan di repositori publik.

c. Build & Push ke GHCR: Job deploy membangun image Docker backend dan frontend, kemudian mengunggahnya ke GitHub Container Registry (ghcr.io). Tag image menggunakan nomor commit SHA pendek (sha_short), bukan tag latest, untuk menjamin keterlacakan versi build.

6. Dockerfile Multi-Stage & Keamanan Kontainer (Soal 5 - CPMK 4 & 5)

Dockerfile dirancang mengikuti standar keamanan dan efisiensi kontainerisasi modern:

a. backend/Dockerfile: Menggunakan image dasar resmi FrankenPHP versi spesifik (dunglas/frankenphp:1.4-php8.3). Berkas composer.json dan composer.lock disalin dan dipasang sebelum kode aplikasi disalin, sehingga memanfaatkan Docker Layer Caching secara optimal.

b. frontend/Dockerfile: Menerapkan *multi-stage build*. Tahap pertama (node:20.18-alpine) memasang dependensi dan menghasilkan berkas statis (npm run build). Tahap kedua menyajikan berkas hasil kompilasi menggunakan nginx:1.27-alpine. Image produksi akhir bersih tanpa menyertakan node_modules.

c. Berkas .dockerignore & Keamanan Secrets: Setiap folder dilengkapi berkas .dockerignore untuk mengecualikan berkas sensitif (.env, kredensial) dan folder dependensi lokal.

7. Orkestrasi Docker Compose (Soal 6 - CPMK 6 & 7)

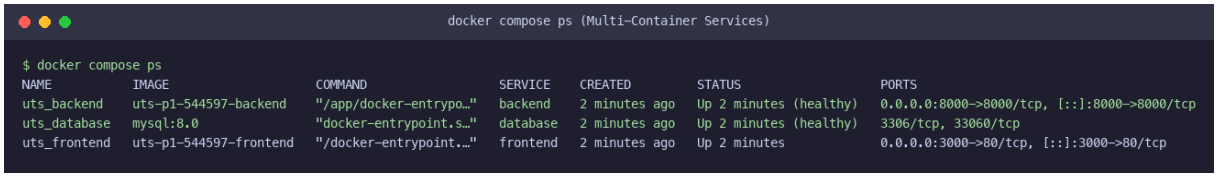
Seluruh sistem berjalan secara utuh dengan satu perintah docker compose up -d --build melalui berkas konfigurasi docker-compose.yml di akar repositori:

a. Tiga Service Terintegrasi: Terdiri atas uts_database (MySQL 8.0), uts_backend (Laravel + FrankenPHP pada port 8000), dan uts_frontend (Vue + Nginx pada port 3000).

b. Service Discovery Internal: Komunikasi antar-kontainer menggunakan nama service dalam Docker Network bridge (uts_toko_network), yaitu backend menghubungi database pada host database:3306 dan Nginx meneruskan request API ke host backend:8000.

c. Persistensi Data Volume: Direktori basis data dipetakan ke Docker named volume uts_toko_db_data. Terbukti bahwa saat kontainer dimatikan dan dihapus dengan docker compose down lalu dinyalakan kembali dengan docker compose up -d, seluruh data produk yang dibuat tetap tersimpan utuh.

d. Variabel Lingkungan: Kredensial dibaca dari berkas .env lokal yang tidak di-commit ke Git. Disediakan berkas .env.example sebagai panduan konfigurasi.



Gambar 7. Output perintah docker compose ps: ketiga kontainer berstatus aktif dan sehat.

8. Nilai Tambahan (+10): Implementasi FrankenPHP Application Server (Soal 7)

Untuk memenuhi tantangan nilai tambahan, backend Laravel dijalankan menggunakan **FrankenPHP** sebagai server aplikasi, menggantikan server bawaan PHP-FPM / Nginx konvensional atau php artisan serve:

a. Image Dasar Jelas: Menggunakan tag versi dunglas/frankenphp:1.4-php8.3 (bukan latest).

b. Integrasi Compose: Kontainer backend berjalan berdampingan dengan service database dan frontend pada jaringan bridge yang sama.

c. Bukti Eksekusi: Log kontainer backend (docker compose logs backend) menunjukkan server FrankenPHP aktif melayani port 8000 dengan mesin Caddy. Log membuktikan request API dilayani langsung oleh FrankenPHP.



Gambar 8. Bukti log kontainer backend membuktikan eksekusi FrankenPHP dan server Caddy pada port 8000.

9. Penjelasan Alur Aliran Data (Data Flow)

Alur pergerakan data dari pengguna hingga ke penyimpanan basis data berjalan sebagai berikut:

1. Interaksi Pengguna (Browser): Pengguna mengakses antarmuka Vue di http://localhost:3000. Saat formulir tambah produk diisi dan tombol simpan ditekan, fungsi reaktif memicu pustaka Axios Client.

2. Reverse Proxy Nginx: Axios mengirimkan permintaan HTTP POST JSON ke path /api/products pada port 3000. Nginx kontainer frontend menerima permintaan tersebut dan secara transparan meneruskannya ke kontainer backend:8000/api/products melalui jaringan internal Docker (uts_toko_network), mengeliminasi kendala CORS antar host.

3. Pemrosesan FrankenPHP & Laravel Sanctum: Server FrankenPHP menangkap request masuk. Middleware Laravel Sanctum memvalidasi header Authorization: Bearer <token> terhadap tabel personal_access_tokens. Jika token sah, request diteruskan ke ProductController@store.

4. Validasi & Eloquent ORM: Controller memvalidasi tipe data, keunikan SKU, dan rentang nilai harga serta stok. Setelah validasi lolos, Eloquent ORM menjalankan query INSERT ke service database:3306 (MySQL).

5. Persistensi Named Volume: MySQL menulis rekaman data ke dalam filesystem kontainer yang terikat pada Docker named volume uts_toko_db_data di host fisik.

6. Respons JSON & Reactive Update: Laravel merespons dengan status **201 Created** dan payload JSON produk baru. Axios menerima data tersebut, memperbarui array reaktif products.value di Vue, dan tabel UI secara otomatis ter-refresh.

10. Kendala yang Ditemui dan Solusi Penanganannya

No	Kendala Teknis	Penyebab Masalah	Solusi Penanganan
1	CORS & Panggilan API Lintas Port	Browser mengakses frontend di port 3000 sementara API di port 8000, menyebabkan pemblokiran Cross-O	Mengonfigurasi Nginx di kontainer frontend sebagai r
2	Permission Storage Laravel dalam Kontainer	FrankenPHP berjalan di bawah pengguna non-root, sehingga gagal menulis log atau cache.	Memvalidasi kepemilikan direktori storage dan bo
3	Docker Layer Caching saat Build	Perubahan kecil pada kode aplikasi memicu instalasi ulang seluruh dependensi.	Memisahkan COPY composer.json dan npm package.json sebelum COPY . .
4	Status 401 saat Permintaan Tanpa Token	Laravel default mengarahkan unauthenticated request ke route 'login', yang tidak ada pada API murni.	Menangkap exception secara eksplisit di bootstrap/app.php untuk mengembalikan respons JSON 401.